

effector — API & Examples Cookbook

Global effects, heterogeneity, importance, regional effects (`find_regions`), and the one-click report

effector v0.4+ (new API)

2026-07-08

Contents

1. What effector is (and the new API in one page)	1
2. The data contract (numpy-only)	2
3. Global effects	3
4. Heterogeneity	3
5. Importance	4
6. Regional effects: <code>find_regions</code> -> <code>Partition</code>	4
7. One-click report: <code>effector.explain</code> -> <code>Report</code>	5
8. Recipes	5
9. Public API index	6

1. What effector is (and the new API in one page)

effector explains black-box models with **feature-effect** methods: global effects (PDP, d-PDP, ALE, RHALE, SHAP-DP), their **heterogeneity**, per-feature **importance**, and **regional effects** (heterogeneity-reducing sub-regions). Everything is built on one lifecycle:

```
construct(data, model[, model_jac], schema=...) # numpy in, numpy out
-> fit(features)                                # the single model touch
-> eval / eval_heter / heter_score / importance / plot # model-free queries
-> find_regions(feature) -> Partition             # regional, a value object
```

Two high-level entry points sit on top:

- `effect.importances()` — rank features by effect strength.
- `effector.explain(data, model, ...) -> Report` — one call: fit, rank, plot the important features, and `find_regions` on the heterogeneous ones; returns a serializable `Report` with a self-contained `to_html()`.

What changed vs. older effector. The `RegionalPDP` / `RegionalALE` / `RegionalRHALE` / `RegionalShapDP` / `RegionalDerPDP` classes were **removed**. Regional questions are now a **query on the global effect**: `global_effect.find_regions(feature) -> Partition`. A `Partition` is a value (nothing is stored on the effect). `importance/importances` and `effector.explain/Report` are new.

Old (removed)	New
RegionalPDP(...).fit(space_partitioner=P)	PDP(...).fit(); .find_regions(f, finder=P)
reg.summary(features=f)	partition.show()
reg.plot(feature=f, node_idx=i)	partition.plot(i)
reg.tree["feature_f"].nodes + node.info[...]	iterate partition; region.mask/.level/.heterogeneity
— (no importance)	effect.importance(f), effect.importances()
— (no report)	effector.explain(...) -> Report, report.to_html()

A running synthetic model used throughout this cookbook:

```
import numpy as np
import effector

rng = np.random.default_rng(0)
N = 3000
x0 = rng.uniform(-1, 1, N)
x1 = rng.uniform(-1, 1, N)
x2 = rng.integers(0, 2, N).astype(float)    # a binary gate
X = np.stack([x0, x1, x2], axis=1)

def model(X):
    # slope of x0 flips with sign(x1) and switches off when x2 == 1
    gate = np.where(X[:, 2] == 0, 1.0, 0.0)
    return gate * np.where(X[:, 1] > 0, 1.0, -1.0) * X[:, 0] * 3.0 + 0.4 * X[:, 1]

def model_jac(X):
    g = np.zeros_like(X)
    gate = np.where(X[:, 2] == 0, 1.0, 0.0)
    g[:, 0] = gate * np.where(X[:, 1] > 0, 1.0, -1.0) * 3.0
    g[:, 1] = 0.4
    return g

schema = {"feature_names": ["x0", "x1", "x2"],
          "feature_types": ["continuous", "continuous", "nominal"],
          "target_name": "y"}
```

2. The data contract (numpy-only)

effector is **numpy-only** at the border:

- **data** — a 2-D numeric np.ndarray of shape (N, D). A pandas DataFrame is **rejected** with a pointer to from_dataframe.
- **model** — a callable (N, D) -> (N,) (numpy in, numpy out). It is passed through untouched.
- **model_jac** — optional callable (N, D) -> (N, D), the per-feature partial derivatives; required only for the derivative methods **d-PDP** and **RHALE**.
- **schema** — metadata: an effector.Schema or a plain dict with any of feature_names, feature_types, cat_limit, target_name, scale_x_list, scale_y, category_names. Anything omitted is inferred. feature_types values are "continuous", "ordinal", "nominal" (aliases "cont", "cat").

From a DataFrame, extract the matrix + a proposed schema without touching the model:

```
X, schema = effector.from_dataframe(df)    # df has no target column
pdp = effector.PDP(X, model, schema=schema)
```

`nof_instances` (constructor kwarg) subsamples the data used to estimate effects; pass an int, or "all". `random_state` makes the subsample reproducible.

3. Global effects

Five methods, one interface. Construct, `fit`, then `eval` / `plot`.

class	needs <code>model_jac</code>	notes
<code>effector.PDP</code>	no	partial dependence + ICE
<code>effector.DerPDP</code>	yes	derivative-PDP (continuous features only)
<code>effector.ALE</code>	no	accumulated local effects
<code>effector.RHALE</code>	yes	robust/heterogeneity-aware ALE
<code>effector.ShapDP</code>	no	SHAP dependence (uses <code>shap/shapiq</code>)

```
pdp = effector.PDP(X, model, schema=schema, nof_instances="all")
pdp.fit("all") # fit every feature (the single model touch)
y = pdp.eval(0, np.linspace(-1, 1, 100)) # (T,) mean effect at those x-values
pdp.plot(0) # matplotlib figure (ICE cloud + mean)
```

`eval(feature, xs, centering=None, mask=None)` returns the (T,) mean-effect array. `centering`: False (raw), True / "zero_integral" (default for PDP/ALE/ShapDP), or "zero_start". `plot(...)` accepts `heterogeneity="ice"/"std"/True/False`, `centering=`, `y_limits=`, `dy_limits=` (RHALE), `scale_x=`, `scale_y=`, `show_plot=`.

Derivative methods take the jacobian:

```
rhale = effector.RHALE(X, model, model_jac=model_jac, schema=schema, nof_instances="all")
rhale.fit([0, 1], binning_method="dp") # binning: "fixed" | "dp" | "greedy" | "quantile"
rhale.plot(0, heterogeneity="std")
```

```
derpdp = effector.DerPDP(X, model, model_jac=model_jac, schema=schema, nof_instances="all")
derpdp.fit(0)
```

RHALE and DerPDP are derivative methods and do **not** support nominal features (there is no derivative over an unordered category), so fitting the nominal `x2` raises — pass the list of continuous features (`[0, 1]`) instead of "all".

ALE binning is configured with `effector.axis_partitioning` objects or strings:

```
import effector.axis_partitioning as ap
ale = effector.ALE(X, model, schema=schema, nof_instances="all")
ale.fit("all", binning_method=ap.Fixed(nof_bins=20, min_points_per_bin=0))
```

SHAP-DP wraps a SHAP backend; keep `nof_instances` modest because it is the expensive method:

```
shap = effector.ShapDP(X, model, schema=schema, nof_instances=500)
shap.fit("all") # backend="shap" (default) or "shapiq"
shap.plot(0)
```

4. Heterogeneity

Heterogeneity lives on its own surface (never mixed into the mean effect):

- `eval_heter(feature, xs, mask=None)` -> (T,) — the non-negative heterogeneity **curve**.
- `heter_score(feature, mask=None)` -> float — a single number (mean of the curve).
- `plot(..., heterogeneity=...)` — draws it as an ICE cloud ("ice") or a $\pm$ band ("std").

```
xs = np.linspace(-1, 1, 100)
h_curve = pdp.eval_heter(0, xs)      # (T,)
h_score = pdp.heter_score(0)         # float
pdp.plot(0, heterogeneity="ice")
```

Heterogeneity measures the spread of the **per-instance** effects around the mean — it is what regional effects reduce.

5. Importance

`importance(feature, mask=None) -> float >= 0` is the **dispersion of the mean effect** — the μ -twin of `heter_score` (which is the spread of the per-instance effect). It is model-free (re-summarised from cached local effects), centering-invariant (no `centering` kwarg), and data-weighted. `importances(mask=None) -> (D,)` is the whole vector.

```
pdp.fit("all")
print(pdp.importance(0))          # scalar
print(pdp.importances())         # (D,) vector, ranks features by effect strength
```

Per-method meaning:

- **PDP / ALE / RHALE** — std of the centered mean effect over the observed values.
- **ShapDP** — `mean(|phi|)`, the classic mean-absolute SHAP value.
- **DerPDP** — `mean(|derivative|)`.

`importances()` returns NaN (with one `UserWarning`) for features whose type a method does not support (e.g. RHALE on a nominal feature).

Importance and heterogeneity are orthogonal. A feature can be highly heterogeneous yet have low importance if its mean effect cancels out (as `x0` does above): that is exactly the feature a global average hides and a regional analysis reveals.

6. Regional effects: `find_regions -> Partition`

Regional effects are a **query** on the fitted global effect. `find_regions(feature, finder=...)` searches for subregions that minimise heterogeneity and returns a **Partition** — a value object; nothing is stored on the effect.

```
pdp = effector.PDP(X, model, schema=schema, nof_instances="all")
pdp.fit(0)
part = pdp.find_regions(0, finder="best")  # "best" | "best_level_wise" | a Best(...) instance
part.show()                               # prints the partition tree + level stats
```

Configure the search with a finder object:

```
finder = effector.space_partitioning.Best(
    min_heterogeneity_decrease_pcg=0.3,  # min relative heter drop to accept a split
    numerical_features_grid_size=10,     # candidate split positions per feature
    max_depth=2,
)
part = pdp.find_regions(0, finder=finder,
                        candidate_conditioning_features="all")
```

A **Partition** is a container of **Regions**:

```
len(part)          # number of regions (root + splits)
part.leaves        # the leaf regions
for r in part:
```

```

r.idx, r.level, r.mask, r.heterogeneity, r.nof_instances, r.weight

part.mask(2)           # boolean (N,) mask of region 2 (a copy)
part.eval(2, xs)        # mean effect within region 2
part.eval_heter(2, xs)  # heterogeneity within region 2
part.plot(2, heterogeneity="ice", centering=True)  # region idx == old node_idx
part.to_dict()          # fully serializable (no model reference)

```

The heterogeneity a `Partition` reports is `heter_score(feature, mask)` on each region's mask — regional and global are the same computation on different masks.

For real datasets, pass display scaling exactly as before:

```

part = pdp.find_regions(3)           # feature 3, default "best"
part.show(scale_x_list=scale_x_list)
for r in part:
    if r.level == 1:
        part.plot(r.idx, scale_x_list=scale_x_list, scale_y=scale_y)

```

7. One-click report: `effector.explain` -> Report

`explain` runs the whole pipeline in **one model touch**: fit → rank by importance → build top-*k* curves → `find_regions` on the features whose heterogeneity clears a threshold. It returns a `Report` value.

```

report = effector.explain(
    X, model,
    method="pdp",           # "pdp" | "ale" | "rhale" | "shapdp" | "derpdp"
    schema=schema,         # NOTE: explain does not take axis_limits
    top_k=5,
    heter_threshold=None,   # default: median heter_score across ranked features
    finder="best",
    nof_instances="all",
)
report.show()              # importance-ranked table + per-feature partition trees
report.plot_importance()   # horizontal bar chart
report.to_html("report.html") # ONE self-contained page (figures inlined as base64)

```

Pass `model_jac=` for `method="rhale"/"derpdp"`. Because everything after `fit` is model-free, growing `top_k` costs **no extra model calls**.

The `Report` is a value and round-trips without the estimator:

```

from effector import Report
d = report.to_dict()
report2 = Report.from_dict(d)  # unbound: text + importance chart work; live re-plot raises
report2.show()

```

Each `report.features[i]` is a `FeatureReport` with `feature`, `name`, `importance`, `heter_score`, `xs`, `y`, `h`, `partition` (the `partition` is a `to_dict()` if `find_regions` fired, else `None`).

8. Recipes

Rank features, then drill into the most heterogeneous one:

```

pdp = effector.PDP(X, model, schema=schema, nof_instances="all")
pdp.fit("all")

```

```

order = np.argsort(-np.nan_to_num(pdp.importances()))
worst = max(range(X.shape[1]), key=lambda f: pdp.heter_score(f))
part = pdp.find_regions(worst)
part.show()

```

Compare methods on the same feature: build PDP, ALE, RHALE, ShapDP, fit each, and overlay `eval(f, xs)`; or use the `effector.FeatureEffect` facade.

Masked (manual) regional query — any boolean (N,) mask, no search:

```

mask = X[:, 2] == 0
pdp.eval(0, xs, mask=mask)           # mean effect on the sub-population
pdp.heter_score(0, mask=mask)        # its heterogeneity
pdp.importance(0, mask=mask)         # its importance

```

Serialize a report for later / sharing:

```

import json
json.dump(report.to_dict(), open("report.json", "w"))
report.to_html("report.html")

```

9. Public API index

Effect classes (`effector.`): PDP, DerPDP, ALE, RHALE, ShapDP, FeatureEffect. Common methods: `fit`, `eval`, `eval_heter`, `heter_score`, `importance`, `importances`, `plot`, `find_regions`, `payload`.

Values: `Partition`, `Region` (`effector.partition`), `Report`, `FeatureReport` (`effector.report`).

Top-level functions: `effector.explain(...)` -> `Report`, `effector.from_dataframe(df)`, `effector.set_theme(...)`.

Config objects: `effector.Schema`; `effector.axis_partitioning` (`Fixed`, `DynamicProgramming`, `Quantile`, `Agglomerative/Greedy`); `effector.space_partitioning` (`Best`, `BestLevelWise`, `return_default`).

Submodules: `effector.models`, `effector.datasets`, `effector.benchmarks`, `effector.ingestion`, `effector.theme`.

See the online docs (design contracts R1–R13, method semantics, per-notebook examples) for the full reference.